

ENTERPRISE ARCHITECTURE SPECIFICATION

AURORA GEMS

Full-Stack E-Commerce Platform & Engineering Architecture

TECHNICAL BLUEPRINT & AUDIT SPECIFICATION

High-Performance Spring Boot 3 Monolith · PostgreSQL 15 · React 18 · Docker Deployment

Project Name	Aurora Gems Luxury Jewelry Commerce Platform
Architecture Role	Lead Software Architect & Senior Full-Stack Engineer
Technology Stack	Spring Boot 3.3.2, Java 17, PostgreSQL 15, React 18, Vite 5, Nginx
Target Release	v1.0 Production-Ready Build (All Critical & High Issues Resolved)
Security Clearance	Role-Based Access Control (CUSTOMER / OWNER) + HMAC JWT
Subject Code	CS543
Members	Maniya Daxit Ashokbhai (20251651054) Gulshan Kumar (20251651040) Ethasam (20251651038)

Table of Contents

1. Executive Summary & Architecture Topology	3
1.1 System Mission & Engineering Objectives	3
1.2 Four-Tier Architectural Layering.....	3
1.3 Core Engineering Standards & Benchmarks.....	3
2. Microservices & Business Service Specifications	4
2.1 AuthService — Authentication, User Registration & Token Lifecycle.....	4
2.2 ProductService & CollectionService — Merchandising, Search & Ratings.....	4
2.3 OrderService & CartService — Atomic Checkout & Order State Machine.....	4
2.4 CouponService, ReviewService & AdminService — Promotions & Analytics.....	4
2.5 Error Handling & Information Leakage Prevention	4
3. Database Design & Entity Architecture	5
3.1 Relational Entity Catalog (14 Tables)	5
3.2 Performance & Indexing Strategy	5
4. API Specifications & Production Runbook.....	6
4.1 Core REST API Endpoints.....	6
4.2 Production Docker Orchestration & Runbook.....	6

1. Executive Summary & Architecture Topology

1.1 System Mission & Engineering Objectives

Aurora Gems is an enterprise full-stack jewelry commerce platform designed for high-concurrency retail operations. The system decouples customer shopping experiences from owner administration while maintaining shared, authoritative data consistency across stock reservation, payment processing, catalog discovery, and order lifecycle management.

1.2 Four-Tier Architectural Layering

Tier 1 — Client Presentation Layer (React 18 + Vite 5 + Tailwind CSS)

- Customer Storefront (Port 5173 / /): Catalog, faceted search, collections, guest cart sync, reviews, checkout.
- Owner Panel (Port 5174 / /admin): Live revenue pulse, order transitions, coupon creator, review moderation queue.

Tier 2 — Edge Router & Reverse Proxy (Nginx 1.25 Alpine)

- Port 80 Entrypoint: Single-page application fallbacks (try_files for / and /admin/admin.html).
- Performance: Gzip compression, static asset caching, rate-limiting, and /api/ and /uploads/ upstream routing.

Tier 3 — Business Application Layer (Spring Boot 3.3.2 Monolith / Java 17)

- Security Filter Chain: Stateless JWT authentication (15m access, 7d refresh) + dynamic CORS configuration.
- Business Services: AuthService, ProductService, CartService, OrderService, CouponService, PaymentService.
- Observability & Reliability: Spring Boot Actuator (/actuator/health), HikariCP pool, sanitized 500 error handling.

Tier 4 — Persistence & Integration Layer (PostgreSQL 15 + Razorpay + Volume)

- Database: PostgreSQL 15 with pgcrypto UUID keys, native ENUM types, and relational foreign keys.
- Currency Modeling: Strict BIGINT integer paise for zero-rounding-error accounting across all line items.
- External Gateways: Razorpay payment orders with HMAC signature validation and local /uploads persistent storage.

1.3 Core Engineering Standards & Benchmarks

Metric / Standard	Implementation Design	Architectural Impact
Query Complexity	Batch Aggregation (findReviewStatsByProductIds)	Eliminates N+1 queries (2N → 1 query)
Dashboard Compute	Database-level JPQL sum/count aggregations	O(1) memory usage (no stream loads)
Stock Deduction	Atomic @Transactional order placement	Guarantees inventory isolation & safety

2. Microservices & Business Service Specifications

2.1 AuthService — Authentication, User Registration & Token Lifecycle

- Registers customers with unique email validation and BCrypt password encryption.
- Generates short-lived HMAC SHA-256 JWT access tokens (15 minutes) and refresh tokens (7 days).
- Enforces account status checks (disabled users rejected immediately upon login).
- Supports DelegatingPasswordEncoder allowing {bcrypt} and {noop} seed migration.

2.2 ProductService & CollectionService — Merchandising, Search & Aggregated Ratings

- Provides paginated catalog listings filtered by metal, collection, price range, and bestseller tags.
- Solves the classic N+1 query problem by batch-fetching rating averages and review counts in a single query.
- Exposes slug-based product details including metal specifications, images, variants, and approved reviews.
- Supports admin inventory modifications with synchronous stock quantity and price updates.

2.3 OrderService & CartService — Atomic Checkout, Stock Allocation & Order State Machine

- Synchronizes guest local storage carts with authenticated user cart records in PostgreSQL.
- Executes authoritative price calculations on the backend (client-supplied prices are strictly ignored).
- Performs atomic inventory validation and stock decrements inside @Transactional boundaries.
- Manages strict order lifecycle transitions: PLACED → CONFIRMED → SHIPPED → DELIVERED.

2.4 CouponService, ReviewService & AdminService — Promotions, Moderation & Live Analytics

- Evaluates percentage and flat discount coupon validity against minimum order values and max uses.
- Provides verified-buyer review submissions with owner approval and deletion moderation workflows.
- Supplies real-time owner dashboard metrics (gross revenue, orders today, active users, low stock).

2.5 Error Handling & Information Leakage Prevention

- GlobalExceptionHandler intercepts all unhandled exceptions and logs stack traces via SLF4J logger.
- Sanitizes 500 Internal Server Error payloads to prevent leaking SQL schema, table names, or internal paths.
- Returns structured JSON error responses with HTTP status, human-readable message, and timestamp.

3. Database Design & Entity Architecture

3.1 Relational Entity Catalog (14 Tables)

Table Name	Primary Key	Key Columns & Foreign Keys	Domain Purpose
users	UUID (PK)	name, email (UQ), phone, password_hash ...	Stores customer and owner account records
addresses	UUID (PK)	user_id (FK), name, phone, line1, line2 ...	Customer delivery address records
collections	UUID (PK)	name, handle (UQ), description, banner ...	Product collection hierarchy
products	UUID (PK)	primary_collection_id (FK), name, slug ...	Core jewelry items with metadata
product_variants	UUID (PK)	product_id (FK), sku (UQ), colour, size ...	Size and metal colour variants
product_images	UUID (PK)	product_id (FK), image_url, alt_text ...	Multi-image carousel assets
product_attributes	UUID (PK)	product_id (FK), attribute_key, attribute_value ...	Key-value specification pairs
carts	UUID (PK)	user_id (FK, UQ), created_at, updated_at ...	Persistent active shopping cart
cart_items	UUID (PK)	cart_id (FK), product_id (FK), variant ...	Cart line items with customization
coupons	UUID (PK)	code (UQ), discount_type, discount_value ...	Promotional voucher codes
orders	UUID (PK)	user_id (FK), address_id (FK), status ...	Customer orders with authoritative totals
order_items	UUID (PK)	order_id (FK), product_id (FK), variant ...	Immutable line-item snapshots
reviews	UUID (PK)	product_id (FK), user_id (FK), rating ...	Customer feedback with owner moderation
blogs	UUID (PK)	title, slug (UQ), excerpt, content, banner ...	Storefront editorial and content pages

3.2 Performance & Indexing Strategy

- Foreign Key Indexes: `idx_cart_items_product_id`, `idx_orders_address_id`, `idx_reviews_user_id`, `idx_order_items_product_id`.
- Compound Index: `idx_orders_status_created_at` on `orders(status, created_at)` enables instant date/status filtering.
- Catalog Filter Index: `idx_products_active_bestseller` on `products(active, bestseller)` speeds storefront hero sections.
- Unique Constraints: email, product slug, variant sku, collection handle, coupon code, and blog slug prevent collisions.
- Relational Cascades: Automatic ON DELETE CASCADE on item tables prevents orphan records upon deletion.

4. API Specifications & Production Runbook

4.1 Core REST API Endpoints

Method	Endpoint Path	Access	Description
POST	/api/v1/auth/register	PUBLIC	Register customer account with credentials
POST	/api/v1/auth/login	PUBLIC	Authenticate customer and return JWT pair
POST	/api/v1/auth/owner/login	PUBLIC	Authenticate owner account with elevated role
POST	/api/v1/auth/refresh-token	PUBLIC	Rotate expired access token using refresh token
GET	/api/v1/products	PUBLIC	Paginated catalog with review aggregates
GET	/api/v1/products/{slug}	PUBLIC	Product details, variants, images, reviews
POST	/api/v1/cart/items	AUTH	Add item with selected variant to cart
POST	/api/v1/coupons/validate	PUBLIC	Verify discount coupon code eligibility
POST	/api/v1/orders	AUTH	Place order with atomic stock deduction
GET	/api/v1/owner/dashboard	OWNER	Aggregated gross revenue, daily metrics
GET	/api/v1/owner/orders	OWNER	Paginated order management with filters
GET	/actuator/health	PUBLIC	Liveness & readiness probe for orchestration

4.2 Production Docker Orchestration & Runbook

Docker Compose Deployment Instructions

- Single Command Start: docker compose up --build -d
- Single Command Stop: docker compose down

Production URLs

- Customer Storefront: http://localhost (Port 80)
- Owner / Admin Portal: http://localhost/admin (Port 80)
- Backend API Probe: http://localhost:8080/actuator/health

Default Verified Credentials

Account	Credential	Role
Owner Account	maniyadaxit1234@gmail.com / Daxit@2001	OWNER
Customer Account	aarohi@example.com / Password@123	CUSTOMER
Database	postgres / postgres (DB: aurora_gems_db, Port 5432)	—

Note: Credentials listed above are default seed values for demonstration and staging environments. They must be rotated prior to any production deployment.